SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 07

Agents, multi-agent systems, middleware, MCP, guardrails & observability

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@slit.lk | Course web: courseweb.sliit.lk

Today's Roadmap – 3 Hours

Block A 0–55 mins	**From tool-calling to agents** Understanding agent vs. chain architectures and their limitations
Block B 65–115 mins	**Multi-agent systems & coordination patterns** Orchestration, collaboration models, and multi-agent workflows
Block C 125–170 mins	**Middleware, guardrails, observability & evaluation** Managing safety, logging, system tracking, and LLM evaluation frameworks
Wrap 170–180 mins	**Production checklist, capstone & course close + Q&A** Best practices for deployment, final project guidance, and open discussion

Short breaks in between (55-65m & 115-125m) • Questions welcome at any point

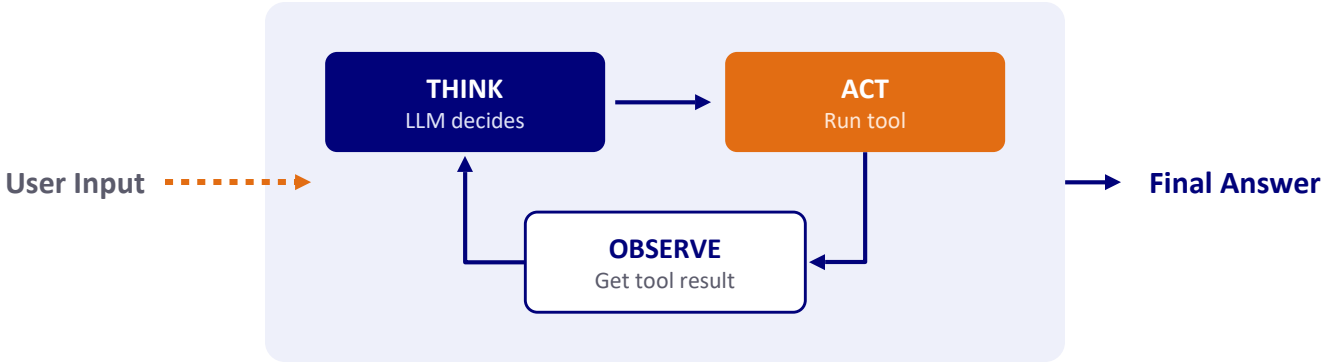
PART 1 OF 4

From tool-calling to agents



What is an agent?

An **agent** is an LLM placed **in a loop** with tools, deciding *which* tool to use, *when*, and with *what arguments* — until it can answer.



Week 2 you ran this loop manually. An agent runs it **for you**, repeatedly, stopping when the model emits a final answer (no more tool requests).

The reasoning pattern (ReAct, conceptually)

Most agents follow a *Reason + Act* rhythm:

Thought: I need the Team plan price.

Action: `search_handbook("Team plan price")`

Observation: *"Team: $25/user/month..."*

Thought: Now compute 8 users * 12 months * 0.8.

Action: `calculate("8 * 25 * 12 * 0.8")`

Observation: *1920*

Thought: I can answer now.

Answer: `"$1,920/year."`

The model interleaves reasoning with actions, using each observation to decide the next step.

That interleaving is what makes it an ***agent***, not a chain.

Tools revisited — the stakes just rose

An agent is only as good as its tools — and this week **no human is in the loop**:

The docstring is the interface

The docstring acts as an interface that helps the agent understand when and how to use the tool.

Week 2's good-tool principles (clear names, typed args, single purpose) were craft advice while *you* closed the loop. With the model choosing **unsupervised**, they become **correctness requirements**.

Retriever as a tool

A retriever can also be provided as a tool.

A retriever can be a tool → the agent decides *if*, *when*, and *how many times* to search. That's the **agentic RAG** teased at the end of Week 2's production block — today you can finally build it.

Same tools as Week 5. What changed is who's choosing — and that changes everything.

Why agents need limits

The Risk: Uncapped Loops

An agent can easily loop: call a tool call another and another ... forever.

Uncapped agents are the **#1 way to burn money** and **hang a live demo**.

How to Bound Them

- **Hard step cap**: max allowable iterations
- **Time budget**: strictly enforce a wall-clock limit
- **Error recovery**: malformed requests shouldn't crash
- **Prompt rules**: "stop and say so after N tries"

> ***"Autonomy without limits isn't intelligence — it's an infinite loop with a credit card."***

Failure modes of agents

Symptom	Cause	Mitigation
Infinite / long loops	No stop condition; ambiguous goal	**Step cap, timeout, clearer prompt**
Wrong tool chosen	Vague tool descriptions	**Better names/docstrings; fewer tools**
Hallucinated tool args	Loose arg schema	**Typed, constrained arguments**
Ignores tool result	Weak prompt; too much context	**Tighter prompt; summarise history**
Runaway cost	Too many tools/steps	**Specialise (multi-agent); cap steps**

> Notice the cure for several rows is **fewer, clearer tools** — which motivates multi-agent design (Block B).

PART 2 OF 4

Multi-agent systems & coordination



Why multi-agent?

The transition from monolithic cognitive systems to modular, collaborative intelligence.

The Monolithic Problem

One single agent loaded with **20+ tools** suffers severe limitations during execution:

- Gets **confused** selecting the right tool
- Becomes **slow** with bloated context lengths
- Extremely **hard to debug** execution pathways

The Multi-Agent Fix

Mirror software engineering practices: **specialise and coordinate** across distinct roles:

- **Researcher**: Equipped with search tools
- **Coder & Writer**: Focused prompt & toolsets
- **Coordinator**: Orchestrates and routes tasks

Mental model: **Each agent receives only the tools and instructions needed for its role. This is similar to microservices, where each service has one clear responsibility.**
Several focused agents can work more effectively than one overloaded agent.

Benefits & costs of going multi-agent

A strategic evaluation of architectural tradeoffs in collaborative cognitive systems.

Architectural Benefits

- **Focused prompts & toolsets:** Leads to highly specialized agents and better, more accurate decisions.
- **Independently testable:** Isolated agent behaviors make execution paths easy to debug and verify.
- **Reusable specialists:** Modular agent roles can be composed and shared across diverse tasks.
- **Separation of concerns:** Decouples complex workflows into manageable, single-responsibility units.

System Costs & Friction

- **More total LLM calls:** Increased delegation leads to higher token consumption and cost.
- **Coordination complexity:** Managing state, routing, and reliable agent interaction adds systemic overhead.
- **Increased latency:** Sequential handoffs and recursive reasoning cycles slow down overall response times.
- **Observability challenge:** Multi-step cognitive pathways are harder to trace without advanced telemetry.

> *Don't reach for multi-agent reflexively. Use it when a single agent is* **demonstrably overloaded** *or when you have* **genuinely distinct specialties**.

Four coordination patterns

Architectural mechanisms for orchestrating multi-agent workflows.

Router

Classify, then dispatch to **one** specialist.

WHEN:
Distinct categories

Pipeline

Output of one feeds the next (research → write → edit).

WHEN:
Fixed stages

Parallel

Run specialists at once, then merge results.

WHEN:
Independent subtasks

Supervisor

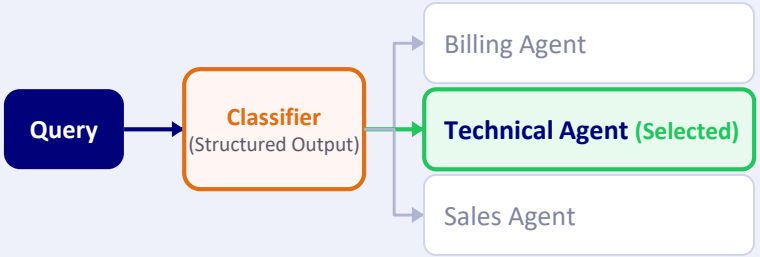
A coordinator calls specialists **as tools**, dynamically.

WHEN:
Unknown, multi-step plan

*The first three patterns normally **follow a predefined workflow**, while the supervisor makes decisions during execution. The supervisor is the only one that's itself an agent.* **Multi-agent ≈ LCEL composition of agents.**

Pattern 1 — Router (dispatch)

A structured-output classifier picks exactly one specialist to handle the incoming request.



Pattern Details

Use when:

Requests cleanly partition into distinct categories.

Strengths:

- **Cheap:** Minimal LLM tokens used.
- **Predictable:** Direct routing logic.
- **Easy to Test:** Evaluate components in isolation.

> ***Classify the request first, and then dispatch it to the correct specialist.***

Pattern 2 — Pipeline (sequential)

Output of one feeds the next — fixed, known stages with no dynamic decisions.

Pipeline Diagram



When to Use

The workflow has clear, ordered, and structured phases with predictable transitions between each specialized task.

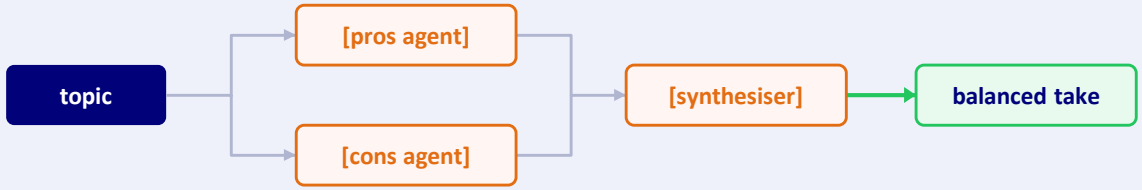
Key Strengths

- **Simple & Predictable:** Sequential execution flow.
- **Plain LCEL:** Next stage strictly consumes the previous output.
- **Independently Improvable:** Optimize each stage in isolation.

> ***If you can define all the stages in advance, a pipeline is usually better than a supervisor agent.***

Pattern 3 — Parallel (fan-out / merge)

Independent subtasks run concurrently, then a step merges them.



Mapping Architecture

Directly maps to Week-1 **parallel + lambda/merge** structures.

When to Use

Apply this pattern when the required subtasks are completely independent of each other and can be executed simultaneously without needing intermediate results from one another.

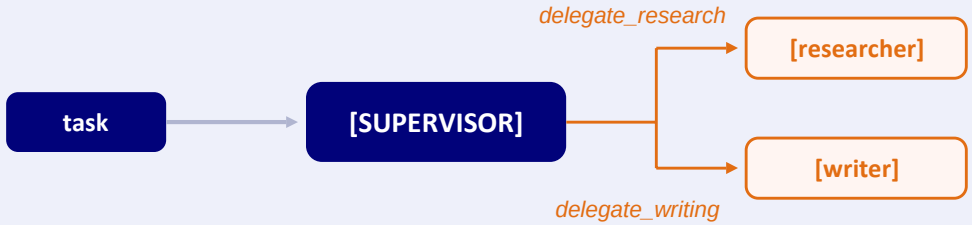
Key Strengths

- **Faster Execution:** Concurrency significantly reduces total latency.
- **Diverse Perspectives:** Allows specialized agents to analyze the same topic from contrasting angles before synthesis.

Run independent tasks together, and then combine their results.

Pattern 4 — Supervisor (orchestrator)

An orchestrator agent dynamically plans and delegates tasks to specialized agent tools.



Agents-as-Tools

A specialist agent becomes just another standard tool that the supervisor dynamically calls.

When to Use

Apply this pattern when the execution plan is unknown up front and requires dynamic, step-by-step orchestration, planning, delegation, and observation over multiple iterations.

Key Strengths

- **Dynamic Flexibility:** Plans and adjusts paths on the fly.
- **Agentic Tools:** Exposes complex sub-agents through a clean tool interface.

The supervisor plans and coordinates while specialists perform focused tasks.

Four multi-agent architectures

Architectural mechanisms for orchestrating multi-agent workflows.

Subagents

A With subagents, one main agent remains in control and calls specialist agents as tools.

All subagent results return to the main agent before the workflow continues.

Handoffs

With handoffs, control can move from one agent to another based on the current state.

For example, a support agent may hand the conversation to a billing agent.

Skills

With skills, one agent stays in control but loads specialized instructions or knowledge only when needed.

Router

With a router, the request is classified and directed to one or more suitable agents.

Their outputs can then be combined into one final response.

Reference: https://docs.langchain.com/oss/python/langchain/multi-agent#handoffs

Choosing a pattern — decision guide

Select the simplest agentic coordination structure that satisfies your architecture requirements:

If your architectural question is...	Then utilize this pattern →
Distinct categories, one handler each?	**Router**
Fixed, ordered stages?	**Pipeline**
Independent subtasks to merge?	**Parallel**
Plan is dynamic / discovered at run time?	**Supervisor**

Strategic Advice: Start at the **simple end** (router/pipeline). Move toward **supervisor** only as the plan becomes genuinely dynamic — you trade predictability and cost for flexibility.

Handoffs — what does the specialist actually see?

A **handoff** is the information passed from one agent to another.

Handoff style	Passes	Trade-off
Full transcript	**everything so far**	max context; max tokens; **distracts the specialist**
Task brief	**goal + constraints + key facts**	cheap & focused; **risks omitting context**
Structured brief	**typed schema** (`task, inputs, output`)	machine-checkable; **needs upfront design**

Treat every handoff like an **API contract**, clearly define what information is passed and what result is expected. **Pass enough context to complete the task, but avoid unnecessary information.**

Shared state — save outputs, pass references

Keep large reports, datasets, and code outside prompts. Share a **file path or record ID** instead.

1. Save and share

- **Save large outputs:** Specialists store reports, datasets, or code in a file, database row, or document store.
- **Return a reference:** Share a path or record ID—such as **research/acme.md**—so another agent opens it only when needed.

2. Why it helps

- **Fewer tokens:** Shorter prompts use fewer tokens and keep other agents focused.
- **Recover and audit:** Saved work survives crashes and makes each handoff easier to inspect and audit.

Key idea: share a **document link** instead of copying the whole document into every message.

Partial failure — follow these four steps

When one agent fails, the supervisor should **follow four clear recovery steps**.

01 / Retry

Try the failed task again with limited attempts and increasing wait times.

02 / Fall Back

Use a backup model, cached result, or a simpler but honest answer.

03 / Re-Plan

Tell the supervisor what failed so it can change the plan or assign another agent.

04 / Fail Loudly

Clearly tell the user when the task failed or the answer is incomplete.

Recovery flow: Retry → Fall Back → Re-Plan → Fail Loudly.

Control cost and latency as the workflow grows

Every result adds to the supervisor's context, so later steps become slower and more expensive.

Model Tiering

Use cheaper models for routing and simple tasks. Reserve stronger models for complex decisions.

Parallel Fan-Out

Run independent agents together. Total latency becomes the longest branch, not the sum.

Summaries Hops

Compress specialist outputs before adding them to the supervisor's context.

Caching

Reuse stable prompts and deterministic tool results instead of computing them again.

Budgets

Set limits for tokens, steps, time, and cost—and stop or reroute when exceeded.

Control context growth early so coordination does not become too slow or expensive.

Least privilege — limit each agent's access

Multi-agent design should separate capabilities as well as responsibilities.



01 / Compartmentalize

Give each agent only the tools and credentials it needs:

- **Researcher:** read-only search
- **Writer**: no unnecessary tools
- No agent: unrestricted shell

02 / Limit the Blast Radius

A compromised agent cannot use tools it never received.

Example: without an email tool, a research agent cannot send messages.

03 / Add Approval Gates

Require approval before sending email, deleting files, making payments, or deploying.

Specialists propose actions; the supervisor or a human authorizes them.

An attack in one compartment should not spread—similar to IAM for each microservice.

Observing & evaluating a multi-agent system

How to gain visibility, measure performance, and establish systematic quality gates:

01 / Traces become trees

Hierarchical Run IDs

A multi-agent execution trace looks like a tree. The supervisor is the parent run, and each specialist creates a child run.

Connected run IDs help us inspect the complete execution path and identify where a failure occurred.

02 / Metrics per agent

Granular Accounting

Track cost, latency, and failure rate **by specialist**. Pinpoint precisely which agent is the slow, expensive, or flaky bottleneck in the system.

03 / Multi-Level Evals

Dual Testing Strategy

- **Component evaluation:** test individual parts, such as router precision, F1 score, or specialist accuracy.
- **End-to-end evaluation:** test whether the complete system successfully finishes real scenarios.

The Multi-Agent Payoff for Testing: Component-level evaluations are what let you **swap one agent's model** and verify that absolutely nothing else broke across the rest of the architecture.

Discussion checkpoint #2 (5 min)

With a neighbour, pick an agent coordination pattern and justify your choice for each scenario:

Scenario 01

Ticket Router: A helpdesk that sends each ticket to billing, tech, or sales.

Pattern tip: Consider a router/supervisor or a classification chain that statically dispatches requests based on intent.

Scenario 02

Blog Factory: Research a topic, draft a post, and then copy-edit it.

Pattern tip: Look at a sequential pipeline or a workflow with dedicated agents passing structured handoffs.

Scenario 03

Dynamic Entry: "Plan and execute a market-entry analysis" — steps unknown.

Pattern tip: Think about a dynamic plan-and-solve agent or supervisor loop that decomposes the goal on the fly.

Discussion Goal: Map these scenarios to specific architectures (e.g., chains, routers, supervisor/worker, or fully autonomous DAG execution) and justify the trade-offs in complexity.

PART 3 OF 4

Middleware, guardrails, observability & evaluation



Cross-cutting concerns

As agents grow, you need behavior that **isn't** the agent's core "thinking". We want them **separated**.

The Problem: Scattered Logic

- Cross-cutting concerns are supporting functions, not the agent's main reasoning.
- Examples include logging, retries, approvals, authentication, redaction, and tracing.
- Adding these functions directly into the agent logic creates complex and messy code.
- It also makes the agent harder to understand, test, reuse, and maintain.

The Solution: Two Clean Mechanisms

1. Middleware: wraps the agent's execution and can modify inputs, outputs, or behavior. (middleware can retry a failed tool call or remove sensitive information.).

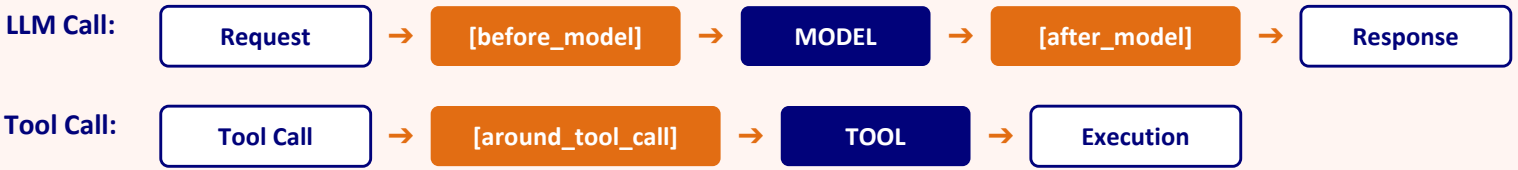
2. Callbacks: observe execution events for monitoring, tracing, and logging. Callbacks normally observe the process without changing it.

Middleware wraps and modifies execution steps directly, while **Callbacks** provide read-only observability to monitor events as they happen.

Middleware — concerns *around* the loop

Middleware works around the agent loop, similar to middleware in web applications. It adds supporting behaviour without rewriting the agent's main reasoning logic.

Visualizing the Loop Wrapper



Execution Hooks

before model call (each LLM step)
→ it can add context, trim old history, or validate the request.

after model call (each LLM step)
→ it can log, validate, or remove sensitive information from the response.

around tool call (each tool run)
→ It can handle authorization, retries, caching, timeouts, and sandboxing.

run boundaries (start / end)
→ It can collect performance metrics and auditing information.

Key Takeaway

Middleware keeps operational tasks separate from the agent's decision-making process.

Middleware in practice (concept)

Middleware can be either built-in or custom-made.

Built-in Middleware

provides common features that can be used immediately.

- **History Summarization:** automatically shortens old conversation history to reduce token usage.
- **Human-in-the-Loop:** pauses risky actions and requests approval from a person.
- **Guardrails & Retries:** detect errors, enforce safety rules, and retry failed operations.

Custom Middleware

is created for the specific requirements of an application.

- **before_model:** checks or modifies the prompt before it reaches the model.
- **wrap_tool_call:** checks tool parameters and prevents unauthorized actions..

Middleware keeps safety, cost control, and logging separate from the main business logic.

Guardrails — layered safety

Guardrails are safety controls placed around an AI agent.Guardrails apply at **three layers**:

01 / INPUT

Input Layer

Reject prompt-injection, off-topic, or unsafe requests **before** the model runs.

02 / TOOL

Tool Layer

Allow-list tools, validate arguments, sandbox execution, and require **human approval** for destructive actions.

03 / OUTPUT

Output Layer

validate the response format, remove sensitive data, and prevent **secret leakage.**

Golden Rule: Never let an agent run arbitrary shell, SQL, or `eval` from raw model text. Restrict what the agent is allowed to do instead of simply trusting it.

Observability — you can't fix what you can't see



- Observability means understanding what happens inside the agent while it is running.
- The final answer alone does not show how the agent reached its decision.
- Important hidden details include the selected tools, arguments, model calls, errors, and repeated loops

Two complementary lenses:

Callbacks are triggered when specific events occur.

- ***Examples*** include model start/end, tool start/end, errors, and token streaming.
- Callbacks are useful for logging, cost tracking, monitoring, and displaying live progress.

Tracing records the agent's complete execution path from start to finish.

- Tools such as LangSmith can visually display every step taken by the agent.

Difference: A callback responds to an individual event, while a trace connects all events into one complete journey.
If we can clearly observe the agent's actions, we can debug problems much faster.

PART 4 OF 4

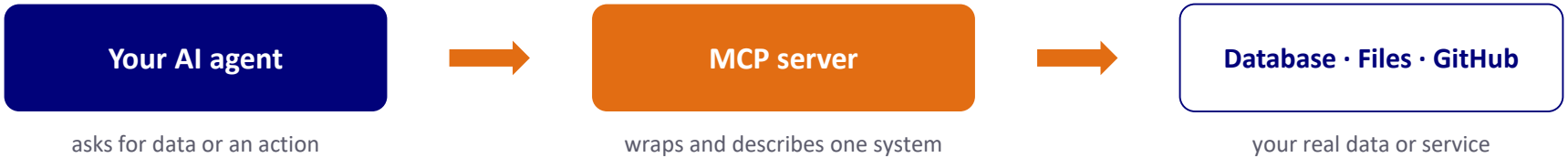
Model Context Protocol (MCP) Servers



What Is an MCP Server?

The one idea to hold on to before we look at any architecture.

An MCP server is a **small program that wraps one system** and exposes it to **any AI application** through **one shared protocol**.



Everyday analogy: a power socket. Any appliance with the right plug just works — and it never needs to know how the power station runs.

The M × N Integration Problem

MCP reduces the custom work needed to connect AI applications with tools and data.

Without MCP (M × N)

- **Separate connectors:** Each AI app needs its own link to every tool or data source.
- **Example: 3 × 4 = 12** custom integrations for 3 AI apps and 4 tool/data services.
- **Duplicated code:** Similar adapters are rewritten.
- **Inconsistent security:** Each integration handles access differently.
- **High maintenance:** One API change can break several integrations.

With MCP (M + N)

- **Standard interface:** AI apps and servers use the same protocol.
- **Example: 3 + 4 = 7** reusable components: 3 app integrations + 4 server integrations.
- **Reuse & discovery:** Apps discover tools dynamically and reuse compatible servers.
- **Consistent controls:** Hosts can apply permissions, authentication and human approval.
- **Less maintenance:** Reusable integrations reduce duplicated work.

Think USB-C: One common interface for many systems. MCP makes integration simpler and tools easier to reuse and maintain.

MCP Architecture

*MCP architecture contains three main components: **Host, Client**, and **Server.** A decoupled, stateful client-server topology communicating via JSON-RPC 2.0.*

01 / MCP HOST

The User Application

Such as Claude Desktop, Cursor, VS Code, or a custom Python/TypeScript agent runner.

Owns the LLM: Manages reasoning loops, model API keys, conversation history, and user UI.

Security Authority: Intercepts tool calls, enforces permission allow-lists, and requests user approval before execution.

02 / MCP CLIENT

Protocol Connector

is located inside the host and acts as the protocol connector; maintains 1:1 or 1:Many connections to MCP servers.

Lifecycle Manager: Spawns server subprocesses or establishes network SSE streams; conducts handshakes.

Capability Registry: Aggregates schemas from all connected servers and presents them cleanly to the LLM.

03 / MCP SERVER

Domain Specialist

Lightweight external program exposing specific tools, resources, and prompt templates.

Model-Agnostic: Zero dependency on LLM providers; purely executes JSON-RPC requests.

Isolated Boundaries: Encapsulates database drivers, native binaries, and private API credentials securely.

Protocol Dataflow: Host (LLM + UI) ⇄ MCP Client (1:Many Routing) ⇄ [JSON-RPC 2.0 / stdio or SSE] ⇄ MCP Servers (DB, Files, APIs) . The LLM does not directly access external systems; the MCP architecture controls and separates each responsibility.

Three Core Primitives — Resources, Prompts & Tools

MCP standardizes data exchange and execution into three strictly typed primitives.

01 / RESOURCES

GET / Read-Only Context

provide information that the AI can read and use as context. ***Examples*** include files, database schemas, logs, and documentation.

URI-based addressing: e.g., postgres://schema/public, file:///app/config.json.

Dynamic Subscriptions: servers can notify clients when resource contents update.

Risk Level: Low (read-only data)

02 / PROMPTS

Interactive Templates

are reusable instruction templates or predefined workflows. They can accept **parameters**, such as a query to analyse or an error to debug.

User Triggers: Prompts are normally selected or triggered by the user.

Risk Level: Low (guided user interaction)

03 / TOOLS

POST / Model Actions

are functions that allow the AI to perform actions. ***Examples*** include sending an email, updating a database, or creating a file.

JSON Schema: Tool parameters are defined using a schema so the inputs can be validated.

High Impact: requires host guardrails and human approval gates for writes.

Risk Level: High (autonomous action)

Summary: Resources provide data, prompts provide guidance, and tools perform actions.

SE3090 – Software Engineering Frameworks

Lecture 07 • Closing

Questions & Discussion

Thank you

Next: Lecture 08 — Git, Collaborative Development, CI/CD, Security & Code Quality